LAM Engineering Handbook

Oct 18, 2026 · @Mr.Forever

Mandatory rules, development lifecycle, testing standards and release gates for every engineer working on the standalone LAM (Learning Adaptation Model) repository.

1. Engineering mission and non-negotiable principles

The objective is not simply to build an API that returns recommendations. It is to build a reliable, testable, auditable learning-adaptation system that improves a student's ability to learn independently over time.

The LAM PRD establishes the core architecture: an independent service, persistent per-concept learner state, diagnosis, adaptive decisions, retrieval scheduling, versioned APIs and a separately governed shared-model pipeline. This handbook expands those requirements into engineering rules, development milestones, manual testing procedures and release criteria.

Important distinction: rules marked mandatory are proposed engineering standards for the team. Milestone targets and evaluation thresholds must be agreed before the relevant implementation begins. They must not be invented after seeing test results.

What LAM is responsible for

LAM receives structured evidence about a student's learning and determines how that evidence should change the learner model and influence the next learning action. The learning cycle is: Observe, Diagnose, Teach, Test, Update Learner Model, Select Next Best Action, Revisit Later.

LAM does not own Qrie's classroom interface, lesson rendering, tutor conversation generation, voice playback or school administration. Qrie delivers learning experiences and returns the resulting evidence to LAM.

1.1 Mandatory engineering principles

Rule	Required standard
Correctness before complexity	Implement a deterministic, testable baseline before adding sophisticated ML or distributed infrastructure

Rule	Required standard
Evidence before inference	State updates and recommendations must be traceable to evidence, a versioned policy, or an identified model
Unknown is not false	Missing, invalid or unavailable signals must not silently become negative evidence
Assisted is not independent	Success after hints or worked examples must not automatically establish independent mastery
No forced diagnosis	A wrong answer does not automatically establish a misconception
Student isolation	Authorization must be checked server-side on every learner-data operation
No hidden training	Live learning events must not automatically update shared-model weights
Reproducibility	Deterministic components must produce consistent results from the same state, evidence and policy versions
Versioned behavior	APIs, schemas, decision policies, scheduling algorithms and models must be versioned
Fail safely	Missing dependencies and insufficient evidence must lead to defined fallback behavior or explicit errors
Privacy by default	Minimize personal data, control access, and prevent sensitive information from leaking into logs
Measure actual learning	Evaluate independent mastery, retention, transfer and learning time, not just engagement or immediate answer correctness

These principles apply to all modules, pull requests, test environments and production releases.

1.2 What engineers must never do

- Count a correct answer after assistance as conclusive evidence of independent mastery.
- Update learner state using unvalidated or unauthenticated events.

- Infer a permanent learner type, such as "visual learner", from a few observations.
- Declare a misconception based on a single ambiguous mistake.
- Hardcode recommendations that ignore prerequisite constraints.
- Silently discard events or overwrite historical evidence.
- Train shared-model weights from every incoming answer.
- Expose one student's records to another student or tenant.
- Claim a feature is complete because its API returns a successful response.
- Mark a milestone complete without satisfying its acceptance criteria.
- Promote a model or policy without a documented evaluation against a baseline.

2. System boundaries and architectural rules

LAM begins as a modular, independently deployable service. Do not split it into many microservices unless real requirements justify the additional operational complexity.

Internal responsibilities

Component	Responsibility
Qrie / other learning applications	Learning content, student interaction, tutor, assessment delivery
API and authorization	Identity validation, tenant access, schemas, rate limits
Evidence ledger	Validated events, provenance, idempotency, durable history
Learner state engine	Mastery, uncertainty, assisted/unassisted evidence, retention
Diagnosis engine	Misconception hypotheses and diagnostic evidence
Adaptive policy engine	Next action, assistance level, prerequisite constraints
Retrieval and strategy engine	Review scheduling and intervention outcomes
Recommendation contract	Action, target concept, assistance level, evidence, rationale, uncertainty, versions
Shared-model pipeline (separate)	Approved data, offline training and evaluation, versioned model, controlled rollout

Requests flow from the application through authorization into the ledger, then through the state, diagnosis and policy engines to the recommendation contract. The shared-model pipeline sits outside this path.

2.1 Dependency rules

- API handlers call application services; they must not contain the core learning algorithms.
- Domain logic must not depend directly on FastAPI, HTTP request objects or ORM sessions.
- Database access must pass through defined persistence interfaces.
- The curriculum graph must be accessed through a stable, versioned contract.
- The recommendation engine must not generate lesson content itself.
- The shared-model pipeline must be independent of live request processing.
- Qrie must consume LAM through its API or SDK rather than importing LAM's internal modules.

Every subsystem must have a documented owner, input contract, output contract, failure behavior and test strategy.

3. Mandatory software development standards

3.1 Code quality

For a Python implementation, the team establishes these minimum rules:

- Use type annotations for public functions, application services and domain interfaces.
- Validate API input using explicit schemas such as Pydantic models.
- Keep business logic in small, testable functions or classes.
- Use explicit domain types for concepts such as assistance level, event type and assessment mode.
- Avoid broad exception handling that hides failures.
- Avoid silent coercion of missing values to zero, false or an arbitrary default.
- Avoid global mutable state and hidden side effects.
- Define units and ranges for every numeric field.
- Use timezone-aware timestamps, normalized to UTC for persistence.
- Document non-obvious algorithms, assumptions and decision rules.

The team chooses and enforces a formatter, linter, type checker and test runner in CI. Ruff, mypy and pytest are suitable choices; the exact tooling is standardized in the repository.

Mandatory rule: no engineer may bypass a required quality gate merely to merge a feature faster.

3.2 Database and event integrity

The database is part of the correctness boundary, not merely a storage layer. Engineers must:

- Use migrations for schema changes.
- Enforce event uniqueness and idempotency at the database level.
- Use transactions when event ingestion and learner-state updates must be atomic.
- Protect state updates against concurrent writes.
- Preserve state versions and event provenance.
- Keep historical evidence distinguishable from derived learner state.
- Define how corrections and invalidated events are represented.
- Test migrations against a real isolated test database.
- Provide a documented recovery strategy for failed migrations and partial operations.

A duplicate event must not produce a duplicate logical learner-state update. A concurrent update must not silently overwrite another update.

3.3 API standards

Every public endpoint must have:

- A documented request schema and response schema.
- Authentication and authorization requirements.
- Defined success and failure status codes.
- Stable error codes.
- Input validation.
- Request or correlation IDs.
- Bounded timeouts and response sizes.
- Retry behavior where appropriate.
- Version compatibility rules.

A successful HTTP status alone is not proof that the underlying learning decision is correct.

3.4 Security and privacy standards

Before real student data is used, the team must establish:

- Server-side authorization for every learner record.
- Tenant isolation.
- Least-privilege service credentials.
- Separate development, test, staging and production configuration.
- Secret management that prevents credentials from entering source control.
- Data minimization and retention rules.
- Consent and shared-training eligibility controls.
- Deletion and correction workflows.
- Redaction of sensitive data from logs.
- An incident response process.

A student ID supplied by a client is an identifier, not proof of authorization.

4. Mandatory development workflow for every feature

Every feature follows the same ten-step engineering cycle.

1. **Requirement definition:** create an issue describing the problem, expected behavior, scope, exclusions and acceptance criteria.
2. **Design and contract:** specify inputs, outputs, state transitions, failure modes, privacy implications and test scenarios before or alongside implementation.
3. **Implementation:** build the smallest testable unit of functionality. Keep the change focused and avoid unrelated refactoring.
4. **Automated testing:** add unit, integration, contract or security tests appropriate to the change.
5. **Local verification:** run formatting, linting, type checking, relevant tests and migration checks.
6. **Manual testing:** execute the documented scenarios against the running implementation and record actual results.
7. **Peer review:** a reviewer other than the author checks correctness, architecture, security, failure handling, tests and documentation.
8. **CI verification:** required automated checks must pass before merge.
9. **Staging verification:** deploy the candidate build and perform smoke, regression and end-to-end checks.
10. **Release and monitoring:** confirm the deployed version, inspect metrics, and verify that no stop condition has been triggered.

A developer must not mark a feature complete just because implementation has stopped. It is complete only when the acceptance criteria and required evidence are satisfied.

5. The complete testing cycle

Testing happens continuously, not only at the end of development.

5.1 Unit testing

Unit tests verify individual rules and functions in isolation. Required areas:

- Event schema validation
- Mastery update rules
- Assisted versus unassisted classification
- Diagnosis hypothesis generation
- Assistance-level transitions
- Prerequisite eligibility
- Retrieval scheduling
- Strategy-effectiveness calculations
- Uncertainty handling
- Recommendation reason codes
- Error and boundary conditions

For every critical algorithm, test normal cases, edge cases, invalid input, missing signals and ambiguous evidence.

Example: given a learner who answers correctly after receiving a hint, the test must verify that assisted performance can change while independent mastery is not automatically promoted solely by that answer.

5.2 Integration testing

Integration tests verify that components work correctly together. They must cover:

- Event ingestion and persistence
- Event ingestion plus state-update transactions
- Duplicate-event handling
- Concurrent state updates
- State retrieval after a process restart
- Database migrations
- Graph adapter integration

- Recommendation generation from persisted state
- Review scheduling after assessments
- Failure recovery after partial operations

Use a real test database for transaction, locking, uniqueness and migration behavior. Mocks alone cannot prove those properties.

5.3 Contract testing

Contract tests verify that LAM's API behaves as documented. They must cover:

- Valid and invalid requests
- Response schemas
- Authentication and authorization
- Error codes
- Pagination
- Version compatibility
- Python SDK behavior
- TypeScript client behavior
- Qrie integration compatibility

A change to a public contract must not silently break an existing consumer.

5.4 Security and privacy testing

At minimum, test:

- Unauthenticated access
- Invalid or expired credentials
- Cross-student access
- Cross-tenant access
- Unauthorized state modifications
- Sensitive data in logs
- Invalid training-data eligibility
- Retention and deletion behavior
- Rate limiting and malformed input

A confirmed cross-tenant data leak is a release blocker regardless of the overall test pass rate.

5.5 End-to-end testing

The complete learning loop is tested as a system:

1. A learner submits an answer.
2. Qrie sends the structured evidence.
3. LAM validates and records the event.
4. LAM updates the learner state.
5. LAM evaluates diagnosis candidates.
6. LAM returns the next action.
7. Qrie delivers the intervention.
8. Qrie reports the outcome.
9. LAM updates the learner state again.
10. LAM schedules a future review when appropriate.
11. A later independent assessment updates the state and review schedule.

This sequence must work across multiple sessions, not only within one API request.

5.6 Manual testing

Manual testing is mandatory for milestone gates and high-risk changes. It supplements automated testing and does not replace it. Use synthetic student data by default.

Each manual test records:

Item	Content
Identifiers	Test ID and requirement ID
Build	Commit SHA and build version
Environment	Environment and database version
Versions	Policy, model and graph versions where relevant
Setup	Preconditions and test data
Procedure	Exact test steps
Expectation	Expected result
Outcome	Actual result and a pass, fail or blocked status
Evidence	Request IDs, redacted logs, screenshots where useful, or database assertions
Defects	Defect ID and retest result

6. Development lifecycle by milestone

Each milestone is independently testable. A milestone cannot pass merely because its code has been written.

M0: Foundation and service contracts

Objective: establish a secure, reproducible, testable service.

Development tasks

- Repository scaffold and module boundaries
- Configuration management
- Authentication and authorization
- Tenant-aware access controls
- Typed event, learner-state and recommendation schemas
- PostgreSQL migrations
- Event ledger and idempotency mechanism
- Versioned API skeleton and SDK skeleton
- CI checks and test infrastructure
- Logging and request correlation

M0 test gate: manual tests

1. Start the service from a clean environment.
2. Create or identify a synthetic authorized learner.
3. Submit a valid event.
4. Submit a malformed event.
5. Try a request without valid authorization.
6. Attempt to access another tenant's learner.
7. Retrieve the stored event and state.
8. Restart the service and verify persistence.
9. Run the migration procedure on a fresh test database.

Pass criteria: contracts are documented, authorization works, migrations are reproducible, event persistence is verified, required CI checks pass, and no unresolved critical or high-severity defect remains.

M1: Deterministic learner model

Objective: implement the first functioning learner-state and decision system.

Development tasks

- Canonical per-concept learner state
- Baseline mastery estimator
- Assisted/unassisted tracking
- Evidence provenance
- Initial misconception taxonomy and rule-based diagnosis
- Initial assistance policy
- Prerequisite-aware recommendations
- State versioning
- Decision reason codes
- State update audit history

M1 test gate: manual tests

1. Submit a first correct independent answer.
2. Submit an incorrect answer.
3. Submit a correct answer after a hint.
4. Submit multiple incorrect answers with ambiguous causes.
5. Submit repeated failure after the same hint.
6. Submit an event with missing optional signals.

7. Submit evidence for multiple concepts.
8. Inspect the resulting state and recommendation.
9. Replay a fixed scenario and verify reproducibility.

Pass criteria: the same valid scenario produces the expected state and recommendation. Assisted evidence remains separate from independent mastery. Weak evidence does not generate unjustified certainty. State changes are traceable to specific evidence.

M2: Closed-loop integration with Qrie

Objective: prove that LAM functions in a real learning workflow.

Development tasks

- Connect Qrie to the versioned API
- Map classroom events to LAM contracts
- Integrate the curriculum graph
- Deliver recommendations through Qrie
- Record the actual intervention delivered
- Record independent assessment outcomes
- Implement retrieval scheduling
- Preserve learner state across sessions
- Add failure and retry behavior
- Add contract and end-to-end tests

M2 test gate: manual tests

1. Begin a learning session.
2. Submit an incorrect answer.
3. Receive and deliver a diagnostic question or appropriate intervention.
4. Submit the resulting response.
5. Verify that learner state changes.
6. Finish the session.
7. Start a second session.
8. Complete an independent assessment.
9. Verify that the new evidence affects mastery and review scheduling.
10. Simulate an API timeout and verify retry safety.

Pass criteria: the entire learning loop works across sessions, Qrie consumes LAM without importing internal modules, and all critical state transitions are verified.

M3: Evidence-based personalization

Objective: make the system adapt based on evidence about which interventions work.

Development tasks

- Strategy-effectiveness estimates
- Confidence and uncertainty calibration
- Proactive intervention triggers
- Metacognitive evidence
- Session-analysis or Learning Scientist component
- Improved policy evaluation
- Delayed retention and transfer tests
- Baseline comparison reports

M3 test gate: manual tests

1. Provide a strategy with insufficient supporting evidence.
2. Verify that the system does not overstate its effectiveness.
3. Provide contradictory intervention outcomes.
4. Verify that uncertainty is retained.
5. Trigger repeated struggle.
6. Verify that the policy selects a defined alternative action.
7. Reprocess the same completed session.
8. Verify that duplicate reviews and state updates are not created.
9. Compare the candidate policy with the baseline.

Pass criteria: personalization decisions are explainable and evaluated. Strategy effectiveness is not inferred from tiny or biased samples. The candidate meets the predefined evaluation thresholds agreed before implementation began.